Coming down to the form, you'll see that I'm hooking the *onsubmit* event to a JavaScript function, and using a JavaScript URL as the action. The idea is to make the form not refresh if I press *Enter* (the JS URL is a quick and dirty way of doing it), and the *onsubmit* event takes care of triggering the appropriate JS function when I submit the form.

Now, let's come down to the script.

First of all, I've created a new *WebSocket* object. For current versions of Firefox, you've got to use *new MozWebSocket* here. Anyway, the WebSocket points to the appropriate URL that we mapped our WebSocket handler to, in the Python server app. Notice the protocol: ws. If you want to use WebSockets over SSL, the protocol is wss.

Just like in the server app, I need to override the *onmessage()* function, which again is an event handler that fires every time the server sends a message. What I've done here is a simple three-step procedure: I created a new HTML paragraph, put whatever the server sent into the paragraph, and appended it inside the big *chatbox div*.

Now let's look at the more interesting DispatchText() function.

1. Here, I first get the contents of the input field and store it in a variable, and immediately clear the field. After I'm done with the field, and provided that the function has been properly coded, you can now use the input field again.
2. Again, I do exactly what I'd done in the *onmessage()* function, displaying what I typed in the chatbox.
3. And then, with the *ws.send()* function, I sent what I typed to the server over the WebSocket.

That's it. WebSockets is this easy to use.

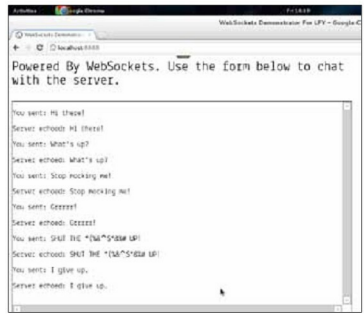
Where to, from here?

Well, now that you know how to use a WebSocket, it's time to leverage this thing.

Here are a few things to remember. A WebSocket is meant to be used with Web applications, and as such, is pretty difficult to use in standard websites with standard tools like PHP and Apache. Remember that applications like Tornado exist so that you can directly expose them to the world without a reverse-proxy in front. If you need to use authentication, you can do that through the WebSocket itself. You don't need to use HTTP 1.1 authentication. You can, but again, it's pretty damed difficult.

If you do need to reverse-proxy a WebSocket, you could use Nginx with a patch, as described in Reverse-Proxying WebSockets with Nginx *(www.letseehere.com/reverse-proxy-web-sockets)*. Check Apache WebSocket (*https://github.com/disconnect/apache-websocket*) if you want to use Apache.

Again, it's the easiest to use a full-blown application server, like Tornado or Node.js. If you want to use Node.js, you can use *Socket.IO*, which is a JavaScript module that works with both Node.js (to implement the server-side) and on browsers (to implement the client side). It has the additional advantage of being able to emulate WebSocket functionality on browsers that don't natively support HTML5 WebSockets—so with *Socket.IO*, you can use WebSockets on IE 5.5 and above, Safari 3 and above, Firefox 5 and above, Chrome 4 and above, as well as Opera 10.61 and above. Check it out at Socket IO (*socket.io/*). You can use the client part of *Socket.IO* without using the server part, so you can use it with Tornado too.



Our little app in action

If you're the geeky type, you can check out WebSockets W3C Specification (*dev.w3.org/html5/websockets/*) for the W3C specification and RFC 6455—The WebSocket Spec (*tools.ietf.org/html/rfc6455*) for the RFC, which defines HTML5 WebSockets. You'll want to explore WebSockets W3C Specification in detail if you're interested in the nitty-gritty on the JavaScript interface to WebSockets. Of course, you don't need to read any of it if you're just going to use *Socket.IO*.

If you're not going to use *Socket.IO*, you can check out CanIUse WebSockets *(caniuse.com/websockets)*, which is continually updated, to see a list of browsers that support WebSockets, along with the level of support they provide.

And finally, you may look at my JS code in the code listing above and laugh, cry or curse me to oblivion. Me, I'm simply waiting for browsers to start natively supporting CoffeeScript. Ha! Check out CoffeeScript *(coffeescript.org/)* if you don't know what that is.

That's all. Now go code. END

By: Boudhayan Gupta

The author likes to describe himself as a complete 'retard by choice'. He believes madness is the cure for everything that is wrong or right with society. He's also a social media enthusiast, and can be reached at @BaloneyGeek over at Twitter.